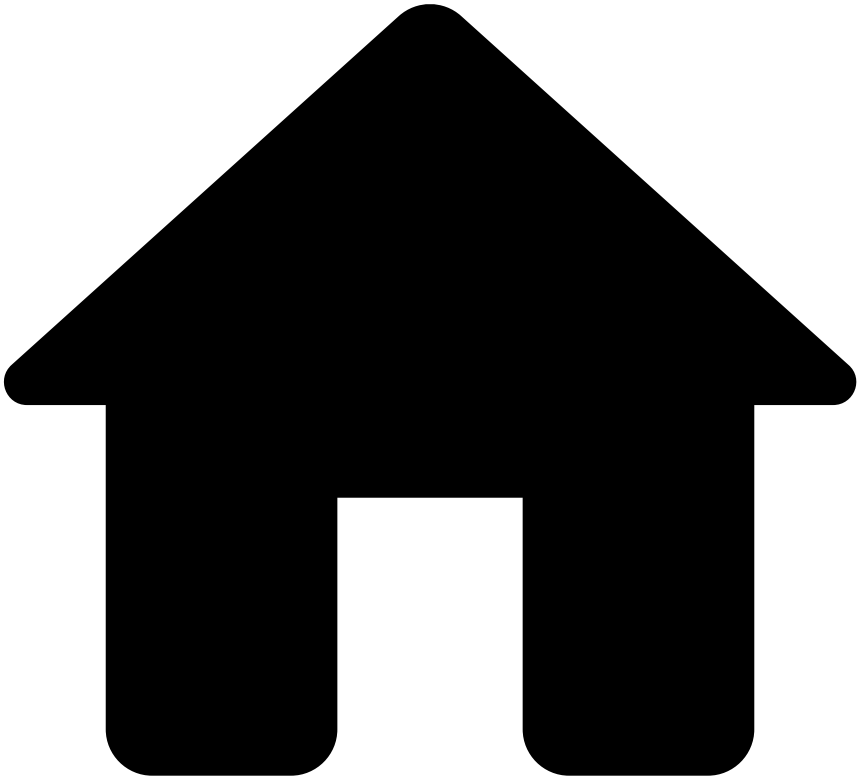


On this page

Was this page helpful?Â 💬

Configuring SSO Using a Generic IdP

Single Sign-On (SSO) in Pulse uses SAML 2.0 standard as the protocol to authenticate and authorize users that are provisioned by an external Identity Provider (IdP).

This guide describes the configuration of SSO using a generic IdP. If you are using Microsoft AD FS as the IdP of choice, please visit the page Configuring AD FS.

Pulse SSO propertiesâ□□

Configuring **SSO** in Pulse requires setting a number of properties. Pulse needs to:

- Know what is the IdP that it will connect.

- Have a private key for signing Pulse requests and a corresponding X509 certificate.
- Have a private key for signing On-Behalf Proxy requests and a corresponding X509 certificate.

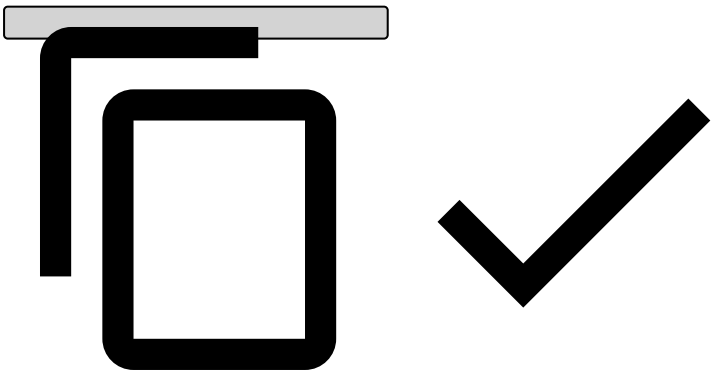
Contact Feedzai to set up these properties for your environment.

Identity provider propertiesâ

The IdP properties have the `pulse.manager.services.core.security.authplugin.filter.samlsso.idp` prefix and the easiest way to know what to include in them is to obtain the **IdP XML metadata**. This XML contains configuration and integration details for SAML 2.0 and is usually made available on a URL. Alternatively, it can be exported as a file on the IdP configuration interface.

The following code block shows the relevant portion of this XML that contains the information that Pulse needs to communicate with the IdP and to which properties the information should be copied.

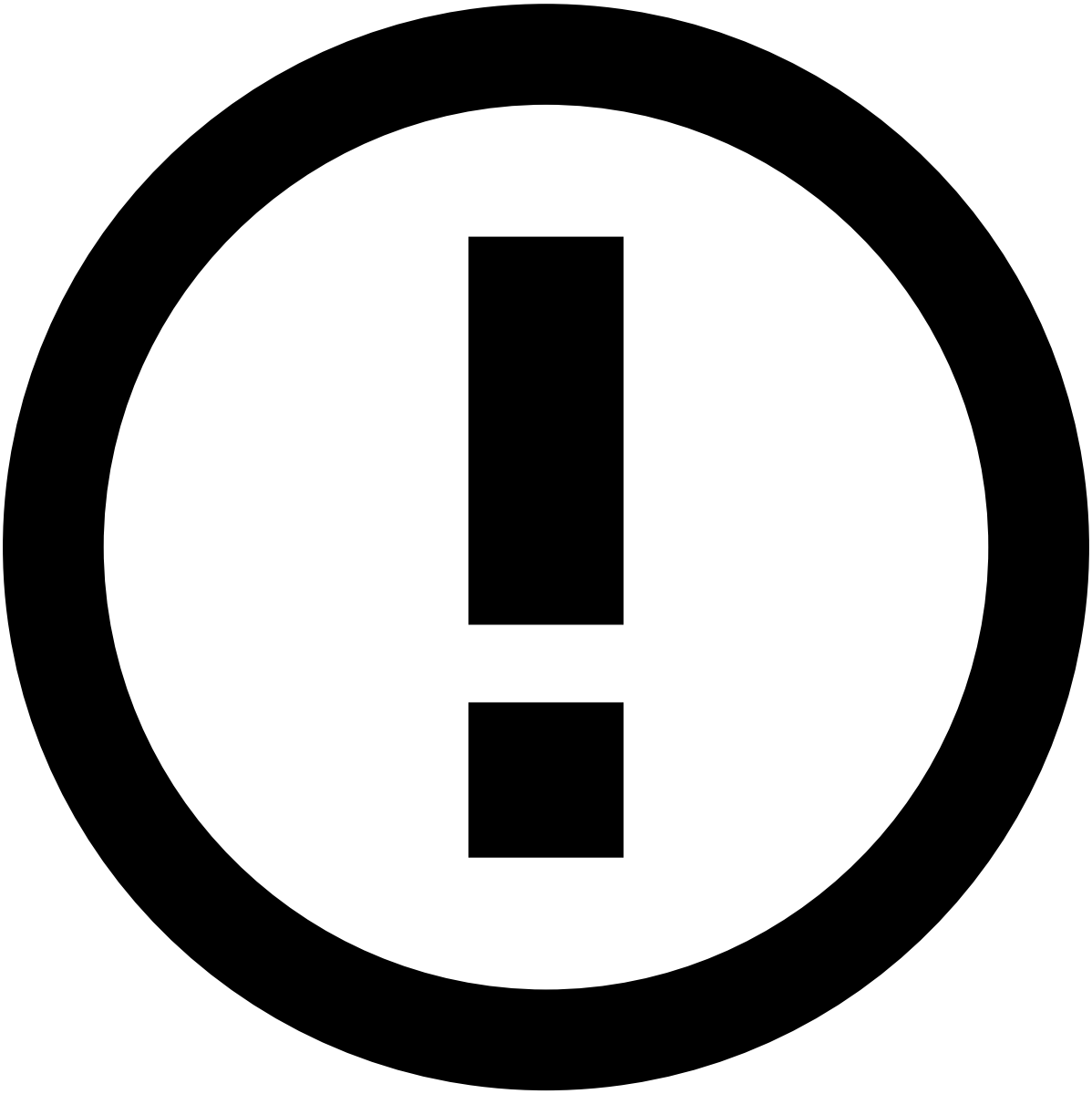
```
 <EntityDescriptor xmlns="urn:oasis:names:tc:SAML:2.0:metadata" ID="..." entityID="{{idp.entityId}
}">
    ...
    <IDPSSODescriptor protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">
        ...
        <KeyDescriptor use="signing">
            <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#">
                <X509Data>
                    <X509Certificate>
                        {{idp.x509cert}}
                    </X509Certificate>
                </X509Data>
            </KeyInfo>
        </KeyDescriptor>
        ...
        <SingleLogoutService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect" Locati
on="{{idp.slo}}"/>
        ...
        <SingleSignOnService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect" Locati
on="{{idp.sso}}"/>
        ...
    </IDPSSODescriptor>
</EntityDescriptor>
```



Service Provider Propertiesâ

The SAML 2.0 standard requires the sign out requests to be signed by the two parties (i.e. Pulse and the IdP). This requires Pulse to have a private key and a X509 public certificate. Optionally, Pulse and IdP can also be configured to sign the Sign in requests.

The following code block exemplifies how to generate the private key and the respective self-signed x509 certificate.

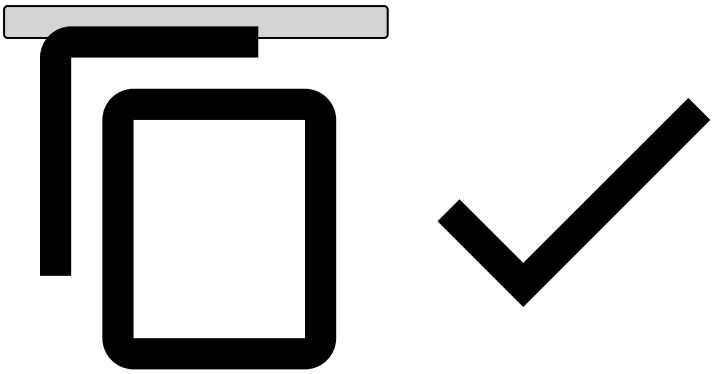


info

Please check if you can use self-signed certificates for **SSO** in your deployment.

```
 # Generate a new Private Key and respective self-signed Certificate, valid for $CERT_DAYS days
openssl req -x509 -nodes -newkey rsa:$KEY_SIZE -keyout $PK_TEMP_FILE -out $CERT_FILE -days $CERT_
DAYS -subj "/C=PT/O=Feedzai/OU=Pulse/CN=feedzai.com"

# Convert the Private Key to PKCS#8 format
openssl pkcs8 -topk8 -inform pem -nocrypt -in $PK_TEMP_FILE -outform pem -out $PK_FILE
```



where the variables are replaced by the following:

Variable	Value	Notes
`$KEY_SIZE`	1024	1024 is the minimum key size required by some IdPs
`$CERT_DAYS`	3650	
`$PK_TEMP_FILE`	pkTempFile.pem	
`$PK_FILE`	pkey.pem	The PKCS#8 private key file.
`$CERT_FILE`	cert.pem	The X509 certificate file.

The contents of `$PK_FILE` and `$CERT_FILE` must be copied to the corresponding properties in Pulse, respectively:

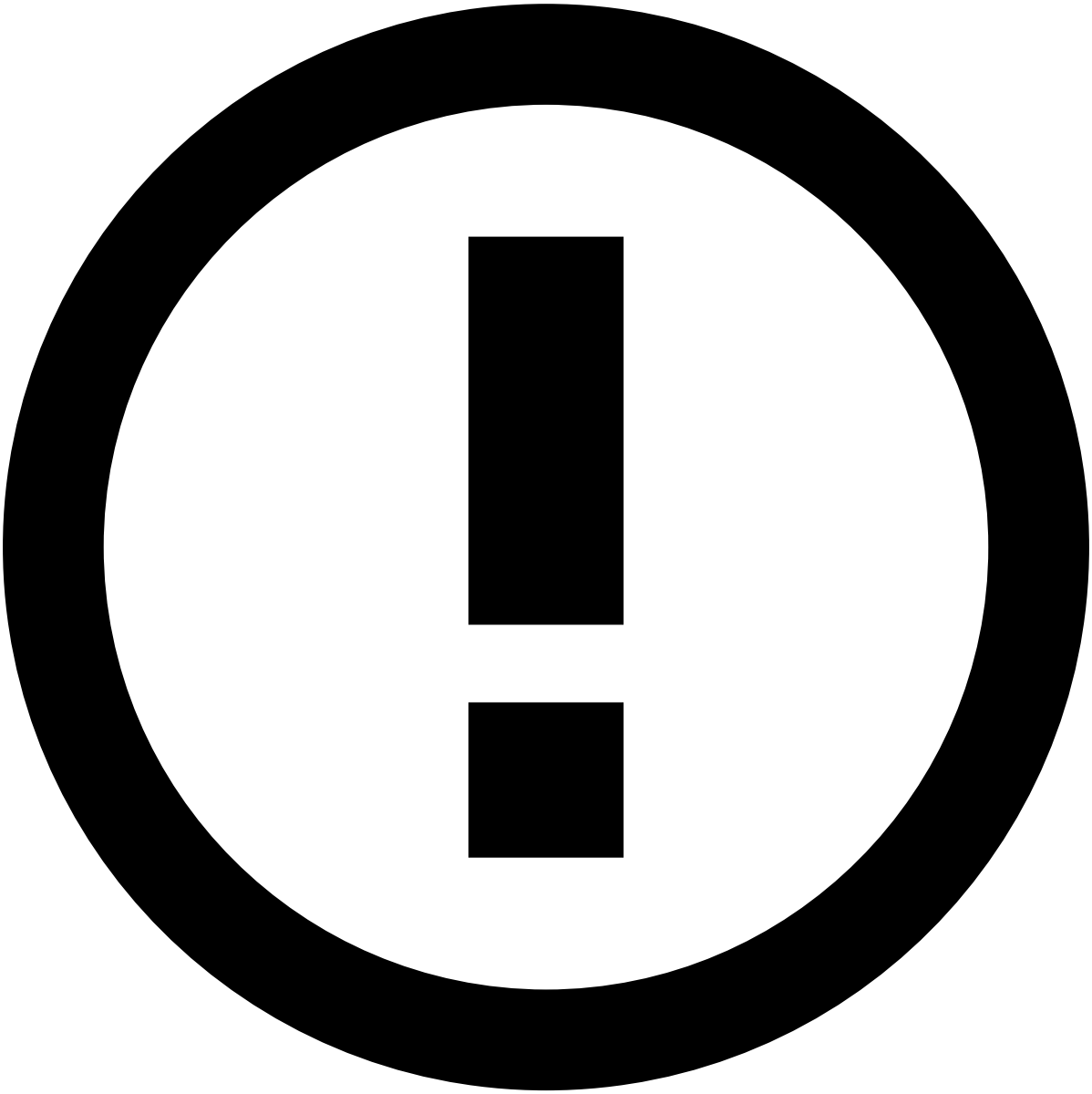
- `pulse.manager.services.core.security.authplugin.filter.samlsso.sp.privateKey`
- `pulse.manager.services.core.security.authplugin.filter.samlsso.sp.x509cert`

On-behalf service provider propertiesâ□□

Pulse is also able to act as a non-standard on-behalf service provider proxy (e.g. Case Manager uses this option).

For each proxy service provider that will use Pulse to handle SAML messages on its behalf, you have to repeat the process of obtaining a private key and respective certificate, but copy the contents of `$PK_FILE` and `$CERT_FILE` to the following Pulse properties instead:

- pulse.manager.services.core.security.authplugin.proxy.samlsso.<spKey>.privateKey
- pulse.manager.services.core.security.authplugin.proxy.samlsso.<spKey>.x509cert



info

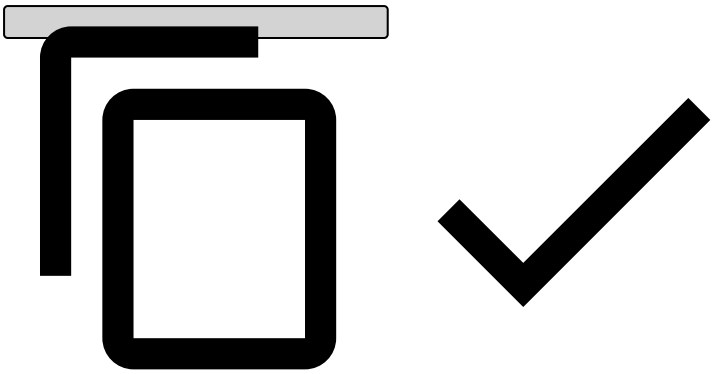
For Case Manager, replace `<spKey>` with `cm`.

Configure IdP to communicate with Pulseâ□□

After configuring Pulse to communicate with the IdP and trust its information, the IdP also needs to be configured to work with Pulse. The SAML 2.0 standard also defines a standard metadata XML for service providers to make their configuration and make their integration details available (e.g. identification, login and logout endpoints, public certificate).

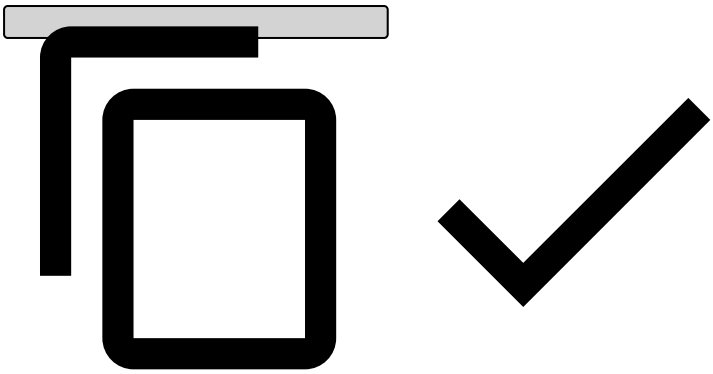
After configuring Pulse for SSO, the XML metadata can be obtained by accessing the following address:

```
https://<BASE_PULSE_URL>/pulseviews/api/sessions/samlsso/metadata
```



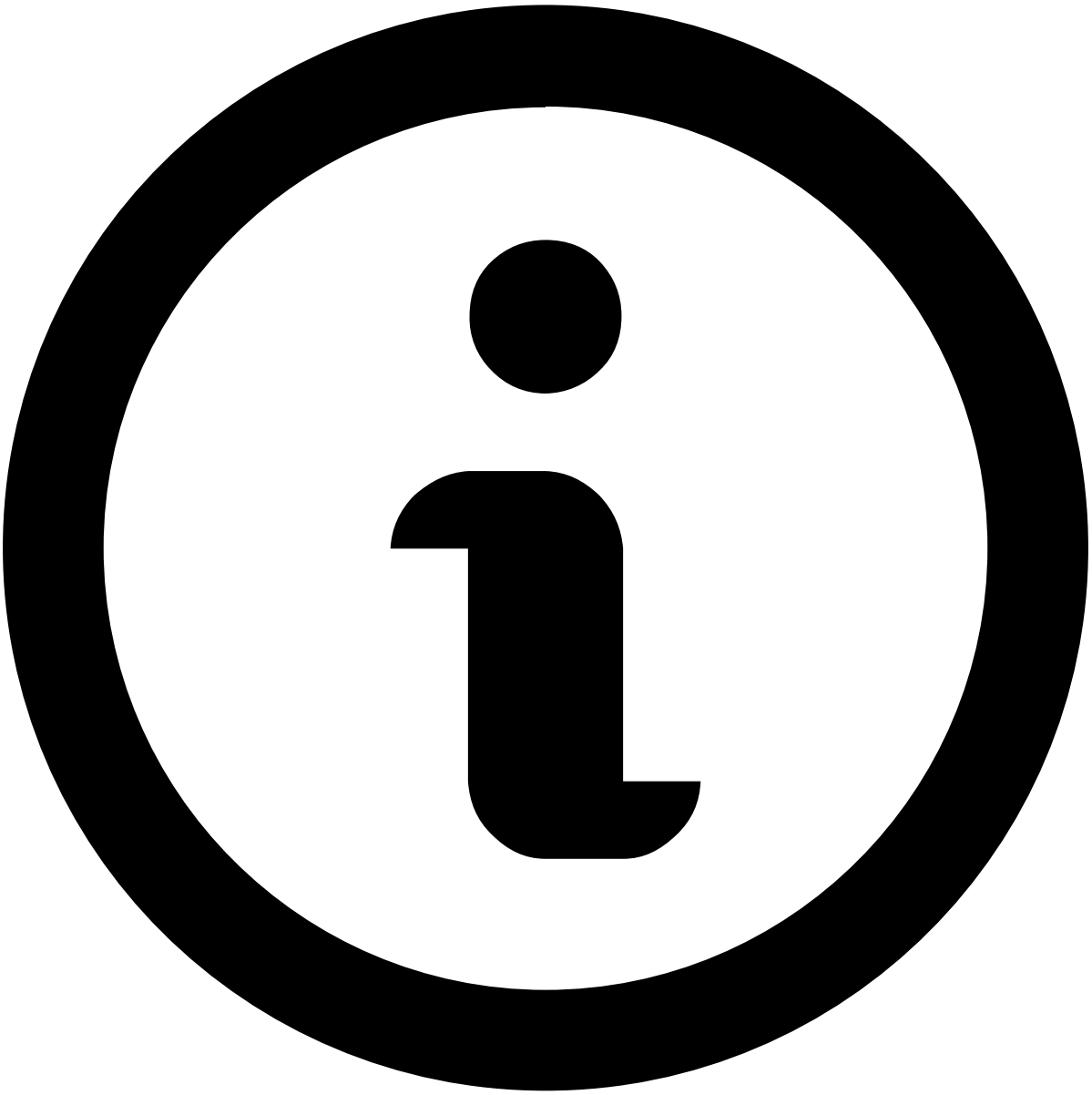
If Pulse is configured to act as an on-behalf service provider (e.g. Case Manager), you can obtain the metadata for the referred service provider by using the following curl request:

```
curl -X POST \
  https://<BASE_PULSE_URL>/pulseviews/api/samlproxy/metadata \
  -H 'Content-Type: application/json' \
  -H 'Cookie: SSOcookie=<A_VALID_PULSE_COOKIE>' \
  -d '{
  "spKey": "<spKey>",
  "baseUrl": "<BASE_HTTPS_URL>",
  "loginEndpoint": "<SAML_LOGIN_ENDPOINT>",
  "logoutEndpoint": "<SAML_LOGOUT_ENDPOINT>"
}'
```



Most IdPs have a method to import these XML metadata files. Even if they don't, this metadata contains all the information that you might need to add Pulse as a trusted service provider on the IdP.

Configure user provisioningâ□□



note

This step is not required if authorization is configured to be performed in Pulse.

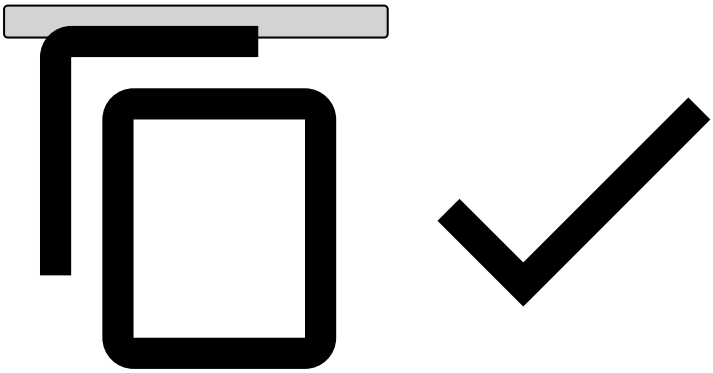
Now that Pulse and the IdP trust and know how to communicate with each other, you can configure user provisioning. Besides the authentication info, Pulse and Case Manager also need to receive authorization information along with users authentication. This information is essential to perform user provisioning (i.e. creating a Pulse user on the fly).

The user provisioning configuration must be set on the IdP configuration interface. Since this configuration is different according to the used IdP, this section focuses on explaining what needs to be configured, as opposed to how to configure it. If Microsoft AD FS is the IdP of choice in your deployment, please visit the page Create claim rules on AD FS.

Pulse performs user provisioning based on the values of SAML attributes within an authentication response. SAML attributes are sent inside the AttributeStatement XML element. Each of these attributes has a name and one or more values.

The following code shows a sample SAML authentication response sent by the IdP to Pulse:

```
 <samlp:Response ID="..." Version="2.0" IssueInstant="..." Destination="https://pulse-local-
signed:8443/pulseviews/api/sessions/samlsso" Consent="..."
    xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
    <Issuer xmlns="urn:oasis:names:tc:SAML:2.0:assertion">{{idp.entityId}}</Issuer>
    <samlp:Status>...</samlp:Status>
    <Assertion ID="..." IssueInstant="..." Version="2.0" xmlns="urn:oasis:names:tc:SAML:
2.0:assertion">
        <Issuer>{{idp.entityId}}</Issuer>
        <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">...</Signature>
        <Subject>
            <NameID Format="...">{{user.username}}</NameID>
            <SubjectConfirmation Method="urn:oasis:names:tc:SAML:2.0:cm:bearer">...</SubjectConf
irmation>
        </Subject>
        <Conditions NotBefore="..." NotOnOrAfter="...">...</Conditions>
        <AttributeStatement>
            ...
            <Attribute Name="User.roles">
                <AttributeValue>UserRole1</AttributeValue>
                <AttributeValue>UserRole2</AttributeValue>
            </Attribute>
            <Attribute Name="User.groups">
                <AttributeValue>UserGroup1</AttributeValue>
                <AttributeValue>UserGroup2</AttributeValue>
            </Attribute>
            <Attribute Name="User.tenants">
                <AttributeValue>UserTenant</AttributeValue>
            </Attribute>
            <Attribute Name="User.language">
                <AttributeValue>en-US</AttributeValue>
            </Attribute>
            <Attribute Name="User.fullname">
                <AttributeValue>User Full Name</AttributeValue>
            </Attribute>
            ...
        </AttributeStatement>
        <AuthnStatement AuthnInstant="..." SessionIndex="...">...</AuthnStatement>
    </Assertion>
</samlp:Response>
```



After receiving the message above, besides the user name Pulse will extract the following information for the authenticated user:

- **Roles:** UserRole1, UserRole2
- **Groups:** UserGroup1, UserGroup2
- **Tenant**: UserTenant
- **Language:** en-US
- **Full Name:** User Full Name

When working with these settings, consider the following:

- Roles, groups and tenants are matched by name. They have to previously exist in Pulse, otherwise the authorization will fail. Exception messages to search for:
  - `Group <name> does not exist`
  - `Role <name> does not exist`
- All SSO-Users must have at least one group or role associated with them. Exception message to search for:
  - `User <username> has no group/role`
- The name of SAML attributes presented above may be changed.
- Besides being created when a user signs into Pulse for the first time, the user information is updated every time the user signs in.
- The content of exchanged SAML messages may be logged. Package: com.onelogin.saml2, Level: debug.